

1. Descripción de la solución

Se desarrolló una aplicación para gestionar materiales bibliotecarios. La solución utiliza una clase base abstracta MaterialBibliotecario y tres clases derivadas: Libro, Revista y DVD. El formulario permite agregar materiales, validarlos, almacenarlos en una lista, buscarlos por título y consultar sus detalles.

2. Clase base MaterialBibliotecario

MaterialBibliotecario contiene las propiedades comunes Titulo, Codigo y Ano. Es abstracta porque representa una idea general de material y define el método abstracto MostrarInfo(), que cada clase hija implementa de forma diferente.

- Titulo: nombre del material.
- Codigo: identificador del material.
- Ano: año del material.
- MostrarInfo(): devuelve la información completa del material.
- ToString(): devuelve un resumen para mostrarlo en la lista.

3. Clases derivadas

Libro hereda de MaterialBibliotecario y agrega Autor.

Revista hereda de MaterialBibliotecario y agrega Numero.

DVD hereda de MaterialBibliotecario y agrega Duracion en minutos.

4. Código de MaterialBibliotecario.cs

```
using System;

namespace Ejercicio4
{
    public abstract class MaterialBibliotecario
    {
        public string Titulo { get; set; }
        public string Codigo { get; set; }
        public int Ano { get; set; }

        public MaterialBibliotecario(string titulo, string codigo, int ano)
        {
            Titulo = titulo;
            Codigo = codigo;
            Ano = ano;
        }

        public abstract string MostrarInfo();

        public override string ToString()
    }
}
```

```

        {
            return $"{GetType().Name}: {Titulo} ({Codigo})";
        }
    }

public class Libro : MaterialBibliotecario
{
    public string Autor { get; set; }

    public Libro(string titulo, string codigo, int ano, string autor)
        : base(titulo, codigo, ano)
    {
        Autor = autor;
    }

    public override string MostrarInfo()
    {
        return $"Tipo: Libro\r\nTítulo: {Titulo}\r\nCódigo: {Codigo}\r\nAño: {Ano}\r\nAutor: {Autor}";
    }
}

public class Revista : MaterialBibliotecario
{
    public int Numero { get; set; }

    public Revista(string titulo, string codigo, int ano, int numero)
        : base(titulo, codigo, ano)
    {
        Numero = numero;
    }

    public override string MostrarInfo()
    {
        return $"Tipo: Revista\r\nTítulo: {Titulo}\r\nCódigo: {Codigo}\r\nAño: {Ano}\r\nNúmero: {Numero}";
    }
}

public class DVD : MaterialBibliotecario
{
    public int Duracion { get; set; }

    public DVD(string titulo, string codigo, int ano, int duracion)
        : base(titulo, codigo, ano)
    {
        Duracion = duracion;
    }

    public override string MostrarInfo()
    {
        return $"Tipo: DVD\r\nTítulo: {Titulo}\r\nCódigo: {Codigo}\r\nAño: {Ano}\r\nDuración: {Duracion} minutos";
    }
}
}

```



```

        break;
    }
}

private void BtnAgregar_Click(object sender, EventArgs e)
{
    try
    {
        var titulo = txtTitulo.Text.Trim();
        var codigo = txtCodigo.Text.Trim();
        var anoText = txtAno.Text.Trim();
        var extra = txtExtra.Text.Trim();

        if (string.IsNullOrEmpty(titulo) || string.IsNullOrEmpty(codigo) ||
            string.IsNullOrEmpty(anoText) || string.IsNullOrEmpty(extra))
        {
            MessageBox.Show("Todos los campos son requeridos.");
            return;
        }

        if (!int.TryParse(anoText, out int ano))
        {
            MessageBox.Show("Año debe ser un número entero.");
            return;
        }

        int anoMin = 1900;
        int anoMax = DateTime.Now.Year;

        if (ano < anoMin || ano > anoMax)
        {
            MessageBox.Show($"Año debe estar entre {anoMin} y {anoMax}.");
            return;
        }

        var tipo = cmbTipo.SelectedItem as string;
        MaterialBibliotecario m = null;

        if (tipo == "Libro")
            m = new Libro(titulo, codigo, ano, extra);
        else if (tipo == "Revista")
        {
            if (!int.TryParse(extra, out int numero) || numero <= 0)
            {
                MessageBox.Show("Número debe ser un entero positivo.");
                return;
            }
            m = new Revista(titulo, codigo, ano, numero);
        }
        else if (tipo == "DVD")
        {
            if (!int.TryParse(extra, out int duracion) || duracion <= 0)
            {
                MessageBox.Show("Duración debe ser un entero positivo.");
                return;
            }
            m = new DVD(titulo, codigo, ano, duracion);
        }

        if (m != null)
        {
            materiales.Add(m);
            lstMateriales.Items.Add(m);
            ClearInputs();
        }
    }
    catch { }
}

```

```

        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ocurrió un error: {ex.Message}");
    }
}

private void ClearInputs()
{
    txtTitulo.Text = "";
    txtCodigo.Text = "";
    txtAño.Text = "";
    txtExtra.Text = "";
    if (cmbTipo.Items.Count > 0) cmbTipo.SelectedIndex = 0;
    txtTitulo.Focus();
}

private void BtnBuscar_Click(object sender, EventArgs e)
{
    var term = txtBuscar.Text.Trim();

    if (string.IsNullOrEmpty(term))
    {
        MessageBox.Show("Ingrese un título para buscar.");
        return;
    }

    for (int i = 0; i < materiales.Count; i++)
    {
        if (string.Equals(materiales[i].Titulo, term,
            StringComparison.OrdinalIgnoreCase))
        {
            lstMateriales.SelectedIndex = i;
            MessageBox.Show(materiales[i].MostrarInfo(),
                "Material encontrado");
            return;
        }
    }

    MessageBox.Show("No se encontró ningún material con ese título.");
}

private void LstMateriales_DoubleClick(object sender, EventArgs e)
{
    if (lstMateriales.SelectedIndex >= 0 &&
        lstMateriales.SelectedIndex < materiales.Count)
    {
        var m = materiales[lstMateriales.SelectedIndex];
        MessageBox.Show(m.MostrarInfo(), "Detalles");
    }
}
}
}

```

7. Interfaz gráfica

La primera imagen corresponde al diseño de la aplicación. Se observan los campos para ingresar los datos generales, el ComboBox para seleccionar el tipo de material, el campo adicional, el botón Agregar, la lista de materiales y el buscador.

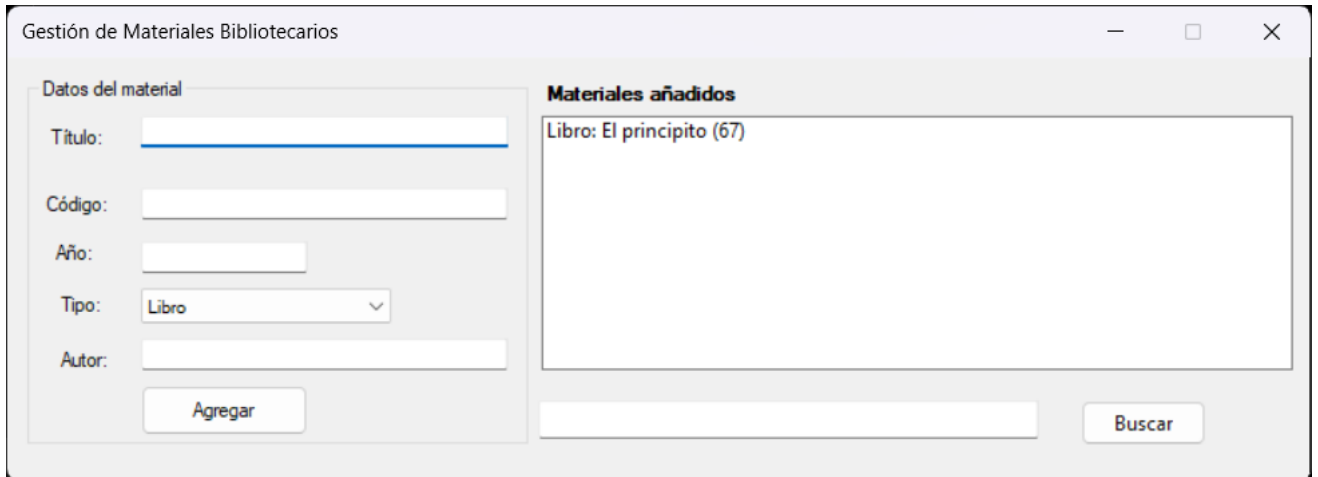
The screenshot shows a Windows-style application window titled "Gestión de Materiales Bibliotecarios". The interface is divided into two main sections. On the left, under the heading "Datos del material", there are five input fields: "Título:" (a long text box), "Código:" (a medium text box), "Año:" (a short text box), "Tipo:" (a dropdown menu currently showing "Libro"), and "Autor:" (a medium text box). Below these fields is a button labeled "Agregar". On the right, under the heading "Materiales añadidos", there is a large empty rectangular area, likely a list or table. At the bottom of the window, there is a search bar (a long text box) and a button labeled "Buscar".

8. Controles utilizados

- txtTitulo – TextBox para ingresar el título.
- txtCodigo – TextBox para ingresar el código.
- txtAño – TextBox para ingresar el año.
- cmbTipo – ComboBox para seleccionar Libro, Revista o DVD.
- lblExtra y txtExtra – muestran y reciben Autor, Número o Duración según el tipo.
- btnAgregar – valida y agrega el material.
- lstMateriales – muestra los materiales agregados.
- txtBuscar – recibe el título que se desea buscar.
- btnBuscar – realiza la búsqueda por título.

9. Corrida de la solución

La segunda imagen muestra la aplicación ejecutándose después de agregar un libro. En la lista aparece 'Libro: El principito (67)'. Este texto se obtiene del método ToString() de la clase MaterialBibliotecario.



10. Resultado de la corrida

- Tipo seleccionado: Libro.
- Título: El principito.
- Código: 67.
- Resultado mostrado: Libro: El principito (67).
- El material fue agregado correctamente a la lista.

11. Conceptos de programación aplicados

En este ejercicio se aplicaron herencia, polimorfismo, abstracción, encapsulamiento mediante propiedades, listas genéricas, constructores, métodos abstractos, validaciones, eventos y controles de Windows Forms.

12. Conclusión

La solución permite gestionar diferentes tipos de materiales bibliotecarios utilizando programación orientada a objetos. La clase base contiene los datos comunes y las clases derivadas agregan sus características específicas. El formulario se encarga de recibir los datos, validarlos, crear los objetos, almacenarlos, buscarlos y mostrar su información.